

南开大学

NANKAI UNIVERSITY, P.R. CHINA 1919

允公允能 日新月异



汇编语言与逆向技术

第5章 过程



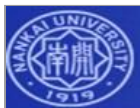
允公允能 日新月异

本章知识点

1. 程序链接和链接库
2. 堆栈机制
3. 过程的定义和使用
4. 程序流程图



南开大学
Nankai University



南开大学

NANKAI UNIVERSITY, P.R. CHINA 1919

允公允能 日新月异



程序链接和链接库

链接库

链接库是一组预先编译的代码集合，通常包含了一系列可复用的函数或程序，目的是为开发者提供完成特定任务的现成工具。这些库可以是动态的（在程序运行时加载）或静态的（在程序编译时直接整合到程序中），它们提供了一种高效的方式来扩展应用程序的功能，无需从头开始编写所有代码。

- `includelib \masm32\lib\masm32.lib`
- `includelib \masm32\lib\kernel32.lib`



PROTO伪指令

- PROTO伪指令用于声明链接库中的过程

- `include \masm32\include\masm32.inc`
- `include \masm32\include\kernel32.inc`

.inc是头文件，对lib的声明，内部由PROTO伪指令构成



允公允能 日新月异

PROTO伪指令

- StdOut
 - StdOut PROTO :DWORD
- StdIn
 - StdIn PROTO :DWORD,:DWORD
- ExitProcess
 - ExitProcess PROTO STDCALL :DWORD

- PROTO伪指令用于过程的声明，当过程使用先于定义时，则必须要声明
- 声明写法：将首行的PROC替换为PROTO即可



南开大学
Nankai University

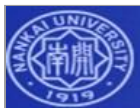
单选题 10分

下面哪个伪指令用于声明链接库中的过程？

- ☐ A PROC
- ☐ B PROTO
- ☐ C SIZEOF
- ☐ D LENGTHOF



南开大学
Nankai University



南开大学

NANKAI UNIVERSITY, P.R. CHINA 1919

允公允能 日新月异



堆栈机制



允公允能 日新月异

运行时栈

- 运行时栈主要有两个基本操作，压栈操作和出栈操作。
- **压栈操作**
 - 压栈操作是将数据元素放入栈顶的过程。在汇编语言中，这通常是通过减少栈指针ESP的值来完成的，因为栈在内存中是向下增长的。每次执行压栈操作时，栈指针首先向下移动一定数量的字节，然后将数据复制到栈指针指向的新位置上。这样可以确保后进入栈的元素始终位于栈顶，可以被最先访问和修改。
- **出栈操作**
 - 出栈操作是从栈中移除元素的过程，通常是指从栈顶开始。在汇编语言中，进行出栈操作时，首先从当前栈指针指向的位置读取数据，然后栈指针会增加相同的字节数，向上移动回到之前的状态。出栈操作后，原先位于栈顶的数据被移除，允许访问在此之前压入栈的下一个元素。



南开大学
Nankai University

使用场景

- **函数调用与返回：**在函数调用时，返回地址和函数的参数通常被压入栈中。当函数执行完成后，通过出栈操作来恢复这些返回地址和参数，保证程序能够返回到正确的位置继续执行。
- **局部变量的存储：**函数的局部变量也经常存储在栈上。这使得每次函数调用时都有一个清晰的局部作用域，函数执行完毕后，这些局部变量可以简单地通过调整栈指针来丢弃。
- **程序执行状态的保存和恢复：**在进行诸如上下文切换这类操作时，程序的当前状态可以被保存到栈中，以便之后可以恢复到相同的状态继续执行。



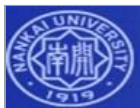
允公允能 日新月异

PUSH、POP指令

- PUSH r/m16
- PUSH r/m32
- PUSH imm32
- POP r/m16
- POP r/m32



南开大学
Nankai University



南开大学

NANKAI UNIVERSITY, P.R. CHINA 1919

允公允能 日新月异



3. 过程的定义和使用

允公允能 日新月异

过程

- C++中的函数定义

```
return_type function_name( parameter list ) {  
    body of the function  
}
```



南开大学
Nankai University



允公允能 日新月异

过程

- **返回类型**：一个函数可以返回一个值。**return_type** 是函数返回值的数据类型。
- **函数名称**：这是函数的实际名称。函数名和参数列表一起构成了函数签名。



南开大学
Nankai University

过程

- **参数**：参数就像是占位符。当函数被调用时，向参数传递一个值，这个值被称为**实际参数**。参数列表包括函数参数的类型、顺序、数量。
- **函数主体**：函数主体包含一组定义函数执行任务的语句。



过程

```
int max(int num1, int num2) {  
    // 局部变量声明  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```





允公允能 日新月异

过程

- 汇编语言中，一般使用术语“**过程**”（procedure）
表示高级语言中的函数、方法
 - 函数（function），C/C++中称为函数
 - 方法（method），JAVA中称为方法



南开大学
Nankai University

允公允能 日新月异

过程

- 汇编语言把过程定义为以返回语句结束的命名语句块。
 - 使用PROC和ENDP伪指令来声明过程
 - 必须定义一个过程名字（标识符）



南开大学
Nankai University

过程

```
my_proc PROC
```

```
... ..
```

```
ret
```

```
my_proc ENDP
```

- 除启动过程之外，其它过程以ret指令结束
 - 将CPU控制权转移到过程被调用的地方



允公允能 日新月异

启动过程（main）

main PROC

... ..

main ENDP

- 启动过程（main）的返回语句是
 - INVOKE ExitProcess, 0
 - 将CPU的控制权转移给Windows操作系统



南开大学
Nankai University

过程的定义

- MyProc过程，计算寄存器EAX、EBX、ECX之和

MyProc **PROC**

ADD EAX, EBX

ADD EAX, ECX

RET

MyProc **ENDP**



允公允能 日新月异

过程的调用与返回

- **CALL**指令将CPU的控制权转移到新的内存地址执行指令，实现过程的调用
- **RET**指令将CPU的控制权返回到程序中过程被调用的地方继续执行



南开大学
Nankai University

允公允能 日新月异

过程的调用与返回

main PROC

MOV EAX, 1000h

MOV EBX, 1000h

MOV ECX, 1000h

CALL MyProc

INVOKE ExitProcess, 0

main ENDP

MyProc PROC

ADD EAX, EBX

ADD EAX, ECX

RET

MyProc ENDP



南开大学
Nankai University

允公允能 日新月异

过程的调用与返回

- 过程返回地址的保存
 - CALL指令调用之后，将过程的返回地址压入堆栈，将过程入口地址赋值给EIP，实现CPU控制权的转移
 - RET指令调用之后，将过程的返回地址赋值给EIP寄存器，实现CPU控制权的转移



南开大学
Nankai University

允公允能 日新月异

过程的调用与返回

00401000	68 00304000	PUSH OFFSET 00403000	ASCII "Hello World", CR, LF	寄存器 (FPU)
00401005	E8 66000000	CALL 00401070		EAX 00003000
0040100A	B8 00100000	MOV EAX, 1000		ECX 00001000
0040100F	BB 00100000	MOV EBX, 1000		EDX 0019FF1C
00401014	B9 00100000	MOV ECX, 1000		EBX 00001000
00401019	E8 1C000000	CALL 0040103A		ESP 0019FF70
0040101E	68 0E304000	PUSH OFFSET 0040300E		EBP 0019FF80
00401023	50	PUSH EAX		ESI 0040103F proc. <ModuleEntry
00401024	E8 1B000000	CALL 00401044		EDI 0040103F proc. <ModuleEntry
00401029	68 0E304000	PUSH OFFSET 0040300E		EIP 0040101E proc. 0040101E
0040102E	E8 3D000000	CALL 00401070		C 0 ES 002B 32Bit 0(FFFFFFFF)
00401033	6A 00	PUSH 0		P 1 CS 0023 32Bit 0(FFFFFFFF)
00401035	E8 E8000000	CALL <JMP.&kernel32.ExitProcess>	跳转至 KERNEL32.ExitProcess	A 0 SS 002B 32Bit 0(FFFFFFFF)
0040103A	03C3	ADD EAX, EBX		Z 0 DS 002B 32Bit 0(FFFFFFFF)
0040103C	03C1	ADD EAX, ECX		S 0 FS 0053 32Bit 308000(FFF)
0040103E	C3	RETN		T 0 GS 002B 32Bit 0(FFFFFFFF)
0040103F	E8 BCFFFFFF	CALL 00401000		D 0
00401044	55	PUSH EBP		O 0 LastErr 00000000 ERROR_S
00401045	BB 00100000	MOV EBP, EBP		
Stack [0019FF6C]=proc.0040101E				
地址	十六进制数据	多字节 (ANSI/OEM - 简体中文)	0019FF70 00401044 0 返回到 proc.00401000 来自 proc.00401	
00403000	48 65 6C 6C 6F 20 57 6F 72 6C 64 0D 0A 00 00 00	Hello World	0019FF74 74CB0419 0 返回到 KERNEL32.74CB0419	



南开大学
Nankai University

多选题 10分

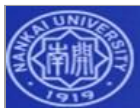
允公允能 日新月异

下面哪个伪指令用于定义过程？

- ☐ A PROC
- ☐ B ENDP
- ☐ C SIZEOF
- ☐ D TYPE



南开大学
Nankai University



南开大学

NANKAI UNIVERSITY, P.R. CHINA 1919

允公允能 日新月异



程序流程图

允公允能 日新月异

程序流程图

- 流程图是一种直观、图形化的表达方法，用于清晰地描述程序、操作或系统的工作流程。它采用一系列符号和带箭头的连线，将复杂的逻辑过程分解成简单、易于理解的步骤。每个图形或符号在流程图中代表一个特定的操作或判断，而箭头则指示了这些步骤之间的执行顺序和流向。



南开大学
Nankai University